

PFPA Summer Intern 2026 — Team 1

Weekly Report | Week 1

Date: June 19, 2026

Team Members: Shenyuan Wu (Team Lead), Guanming Lin, Haixing Tan

Team Overview

This week all three members independently implemented a market data download pipeline for public companies, using Python and the `yfinance` library as the common foundation. A Zoom group meeting was held on Thursday evening (timed to accommodate members based in China) to present and compare results.

Key team decision: Given that each implementation has different strengths and structural choices, the team agreed to keep individual versions separate for now and wait for further guidance from PFPA next week before deciding on a unified structure.

Member 1 — Shenyuan Wu (Team Lead)

Coordination Work

- Contacted all team members individually and set up a **WeChat group** for daily communication
- Created the **GitHub repository** and invited all members
- Scheduled and hosted the **Thursday Zoom group meeting**, coordinating timing across time zones
- Compared individual results during the meeting and facilitated the team decision on next steps
- Collected work summaries from all team members and compiled this weekly report

Technical Work

Developed a Jupyter Notebook (`PFPA_Week1_Data_Download.ipynb`) that automatically downloads and processes all market data required for the KMV credit rating model.

Data downloaded for each company:

Data	Source	Output File
Daily stock price history (OHLCV + dividends)	Yahoo Finance	{TICKER}_price_history.csv
Company info & shares outstanding	Yahoo Finance	{TICKER}_company_info.csv
Daily market cap	Computed: price × shares	{TICKER}_market_cap.csv
Quarterly balance sheet (ST debt, LT debt, KMV Debt D)	Yahoo Finance	{TICKER}_balance_sheet.csv
US Treasury yield (risk-free rate)	Yahoo Finance (^IRX)	risk_free_rates.csv
Merged master dataset	All of the above	{TICKER}_master.csv

Key implementation details:

- KMV Debt D follows the PFPA lecture formula: $D = \text{Short-term Debt} + 0.5 \times \text{Long-term Debt}$
- Stock prices are adjusted by adding back cumulative dividends since the last quarterly report, so E_t reflects total equity return
- Balance sheet data is forward-filled from each quarterly report date to the next, so every trading day carries the most recent available debt figure
- τ is pre-computed for each day and stored in the master dataset, ready for the KMV EM algorithm next week

Tested on: MSFT, AAPL, TSLA (501 trading days, June 2024 - June 2026)

Sample results:

Ticker	Market Equity	KMV Debt D	E/D Ratio
MSFT	\$2.87T	\$32.9B	87×
AAPL	\$4.43T	\$47.5B	93×
TSLA	\$1.50T	\$9.2B	164×

Member 2 — Guanming Lin

Technical Work

Developed a Python command-line script (`download_financial_data.py`) that accepts company names or ticker symbols as input and downloads structured financial data for each company.

Data downloaded for each company:

Data	Output File
Historical stock price (date, open, high, low, close, adj. close, volume)	<code>price_history.csv</code>
Quarterly income statement	<code>quarterly_income_statement.csv</code>
Quarterly balance sheet	<code>quarterly_balance_sheet.csv</code>
Quarterly cash flow statement	<code>quarterly_cash_flow.csv</code>
Market cap history	<code>market_cap_history.csv</code>
Company summary	<code>summary.json</code>
Run manifest (all companies)	<code>manifest.json</code>

Key implementation details:

- Accepts both ticker symbols (e.g. `AAPL`) and full company names as input
- Resolves company names to tickers using the **SEC company ticker database**, with Yahoo Finance search as a fallback
- SEC ticker data is cached within a session to avoid repeated network requests
- Retrieves company metadata including name, exchange, currency, market cap, and shares outstanding
- Market cap is pulled directly from Yahoo Finance; if unavailable, estimated as current price × shares outstanding
- All network requests include timeouts and HTTP error checking
- Missing or unavailable data is handled gracefully without terminating the program
- Output directory structure is consistent even when some data is unavailable (empty files are still created)
- Data values are converted into JSON-compatible formats before saving

- Command-line options allow users to customize the output directory, historical period, and data interval
-

Member 3 — Haixing Tan

Technical Work

Developed a Python script (`download_financial_data.py`) that takes ticker symbols as command-line input and downloads structured financial data for each company via `yfinance`.

Data downloaded for each company:

Data	Output File
Historical stock price (date, open, high, low, close, adj. close, volume)	<code>{TICKER}_price_history.csv</code>
Quarterly income statement	<code>{TICKER}_quarterly_income_statement.csv</code>
Quarterly balance sheet	<code>{TICKER}_quarterly_balance_sheet.csv</code>
Quarterly cash flow statement	<code>{TICKER}_quarterly_cash_flow.csv</code>
Market cap history	<code>{TICKER}_market_cap.csv</code>

All files are saved under a `data/` folder with one subfolder per company (e.g. `data/AAPL/`).

Key implementation details:

- Takes ticker symbols directly as command-line input (e.g. `AAPL`, `MSFT`, `TSLA`)
- Uses `yfinance.Ticker()` to access Yahoo Finance data
- Market cap is pulled directly from Yahoo Finance; if unavailable, estimated as current price $\times$ shares outstanding
- Creates a run summary showing whether each data type was successfully downloaded for each company
- A Jupyter Notebook (`financial_data_w1.ipynb`) is included to demonstrate usage and display sample outputs including price history, quarterly financials, and market cap data
- Project also includes `README.md` and `requirements.txt`

Tested on: AAPL, MSFT, TSLA, NVDA, GOOGL, AMZN

